Московский авиационный институт
(национальный исследовательский университет)

Кафедра 806 «Вычислительная математика и программирование»

Объектно-ориентированное программирование (семестр 1)

01.03.02 «Прикладная математика и информатика»

ЛАБОРАТОРНАЯ РАБОТА 2: КЛАСС С РУЧНЫМ УПРАВЛЕНИЕМ ПАМЯТЬЮ — RAII, ПРАВИЛО ПЯТИ

Общая информация

Параметр	Значение
Срок сдачи	2 недели после выдачи
Prerequisite	Лекции 1–7 (классы, конструкторы, деструкторы, копирование и перемещение)
Язык	C++20
Система сборки	CMake 3.16+ с Google Test

Цели

1. Реализовать класс с ручным управлением динамической памятью.
2. Освоить идиому RAII (Resource Acquisition Is Initialization).
3. Реализовать правило пяти (Rule of Five).
4. Применить идиому copy-and-swap для безопасного присваивания.
5. Написать модульные тесты для всех операций.

Структура работы

Работа выполняется в **трёх семействах** — каждое семейство использует свой класс, но **цели и обязательная база у всех семейств одинаковые**: ручное

управление памятью, RAII, правило пяти, copy-and-swap, методы класса, тесты и проверка утечек.

1. **Семейство A — String**: строковый класс с динамическим массивом char*.
2. **Семейство B — Polynomial**: многочлен одной переменной, коэффициенты double*.
3. **Семейство C — Matrix**: матрица с плоским динамическим буфером double*.

Студент выбирает **семейство**, а затем **номер варианта** внутри него. Базовый интерфейс класса, правило пяти, RAII, copy-and-swap и все методы из раздела своего семейства обязательны для всех вариантов этого семейства. Дополнительно реализуется алгоритм своего варианта и покрывается тестами. При сдаче укажите **семейство и номер варианта**.

Общие требования (для всех семейств)

Запрещено использовать std::string, std::vector, std::array, умные указатели — только ручное управление памятью (new[] / delete[]).

RAII

- Конструктор выделяет память, деструктор освобождает.
- Никаких утечек памяти при любом сценарии использования.

Правило пяти

- Конструктор копирования: глубокое копирование буфера.
- Конструктор перемещения: «обнуляет» источник (O(1)).
- Присваивание: используйте идиому copy-and-swap.
- Деструктор: delete[] буфера.

Copy-and-swap

```
// Оператор присваивания принимает аргумент ПО ЗНАЧЕНИЮ.
// Компилятор сам вызовет конструктор копирования или перемещения.
T& T::operator=(T other) {
    swap(other);
    return *this;
}
```

Память

- Все промежуточные выделения памяти должны быть освобождены.
- Проверьте утечки: cmake -DCMAKE_CXX_FLAGS="-fsanitize=address" ..

Тесты

- Не менее **12 тестов Google Test**: на базовое поведение класса и на алгоритм варианта.
- Тестируйте граничные случаи: пустой объект, один элемент, самоприсваивание, копирование/перемещение, конкатенация с пустым объектом.

Семейство А. Класс String

Строковый класс без использования std::string. Для хранения символов — динамический массив char*.

Стратегия роста буфера

- При push_back / append: если size_ == capacity_, увеличивайте capacity_ в 2 раза.
- Минимальная начальная ёмкость: 1.
- Всегда выделяйте capacity_ + 1 байт для завершающего '\0'.

Примеры использования

```
#include "string.h"
#include <iostream>

int main() {
    String s1;                    // ""
    String s2("Hello");           // "Hello"
    String s3(s2);                // "Hello" (копия)
    String s4(std::move(s3));     // "Hello" (перемещение, s3 теперь пуст)

    s1 = s2;                      // копирующее присваивание
    s1.append(", World!");
    s1.print(std::cout);          // "Hello, World!"

    size_t pos = s1.find("World");
    std::cout << pos << std::endl; // 7
    return 0;
}
```

Варианты семейства A

№	Метод	Алгоритм	Тестовый пример
1	size_t count_occurrences(const String& substr) const	Число непересекающихся вхождений подстроки (наивный проход)	`"abababab".count_occurrences("ab") → 3; "hello".count_occurrences("x") → 0`
2	String remove_char(char c) const	Удалить все вхождения символа (сдвиг влево)	`"banana".remove_char('a') → "bnn"; "abc".remove_char('x') → "abc"`
3	bool is_palindrome() const	Палиндром: сравнение символов с двух концов	`"level".is_palindrome() → true; "hello".is_palindrome() → false`
4	String rle_compress() const и void rle_decompress()	Кодирование длин серий: повторы символа → символ + счётчик	`"AAABCC".rle_compress() → "A3B1C2"; после rle_decompress() — снова "AAABCC"`
5	size_t count_char(char c) const	Счётчик вхождений символа	`"hello".count_char('l') → 2; "hello".count_char('z') → 0`
6	void word_histogram(String* words, size_t* counts, size_t& out_size) const	Разбить строку на слова по пробелам и знакам препинания, посчитать частоты (без повторов)	`"a bb a c a" → {a:3, bb:1, c:1}`
7	String wrap(size_t width) const	Перенос по словам: вставить '\n' так, чтобы строка не разрывалась в середине слова и не превышала width	`"aa bb ccc".wrap(4) → "aa bb\nccc"`

Семейство B. Класс Polynomial

Многочлен одной переменной с действительными коэффициентами. Коэффициенты хранятся в динамическом массиве double*; индекс элемента — степень x.

Инварианты

- Массив всегда имеет размер degree_ + 1.
- В **нормальном виде** коэффициент при x^degree_ не равен нулю (после trim()).
- Нулевой многочлен имеет degree_ == -1.

Примеры использования

```
#include "polynomial.h"
#include <iostream>

int main() {
    double c1[] = {1.0, 2.0};         // 1 + 2x
    double c2[] = {0.0, 1.0, 1.0};    // x + x^2
    Polynomial a(c1, 1);
    Polynomial b(c2, 2);

    Polynomial s(a);
    s.add(b);                         // 1 + 3x + x^2
    s.print(std::cout);
    std::cout << s.evaluate(2.0) << std::endl; // 1 + 6 + 4 = 11

    Polynomial m(a);
    m.mul(b);                         // (1 + 2x)(x + x^2) = x + 3x^2 + 2x^3
    m.print(std::cout);
    return 0;
}
```

Варианты семейства B

№	Метод	Алгоритм	Тестовый пример
8	Polynomial derivative() const и Polynomial integral() const	Производная: coeff[i] * i со сдвигом вниз; интеграл: coeff[i] / (i+1) со сдвигом вверх	`(1 + 2x + 3x^2).derivative()` → `2 + 6x`; `(2x).integral()` → `x^2` (свободный член 0)

№	Метод	Алгоритм	Тестовый пример
9	Polynomial pow(size_t n) const	Итеративное умножение: результат = 1, повторить n раз result.mul(*this)	`(1 + x).pow(3) → 1 + 3x + 3x^2 + x^3; p.pow(0) → 1`
10	double definite_integral(double a, double b) const	Определённый интеграл по [a, b]: первообразная + evaluate	`(x^2).definite_integral(0, 2) → ≈ 2.6667 (8/3)`
11	double find_root(double a, double b) const	Метод бисекции на отрезке [a, b], где знаки на концах разные	`(x^2 - 2).find_root(1, 2) → ≈ 1.4142 (abs(f(root)) < 1e-9)`
12	bool is_equal(const Polynomial& other, double eps = 1e-9) const	Сравнение степеней и каждого коэффициента с точностью eps	`(x^2).is_equal(x^2 + 1e-6, 1e-4) → true; (x).is_equal(x + 1, 1e-4) → false`
13	double coefficient_sum() const	Сумма всех коэффициентов (значение многочлена в точке 1)	`(1 + 2x + 3x^2).coefficient_sum() → 6`
14	void evaluate_many(const double* xs, double* ys, size_t n) const	Таблица значений: для каждой точки вызвать evaluate	`(x^2).evaluate_many({-1,0,1,2}, ys, 4) → ys = {1, 0, 1, 4}`

Семейство C. Класс Matrix

Матрица размером rows × cols, элементы хранятся в одном динамическом массиве double* **построчно**: элемент (r, c) лежит по индексу r*cols_ + c.

Примеры использования

```
#include "matrix.h"
#include <iostream>

int main() {
    Matrix a(2, 2, 1.0);    // [[1,1],[1,1]]
    Matrix b(2, 2, 2.0);    // [[2,2],[2,2]]

    Matrix s(a);
    s.add(b);               // [[3,3],[3,3]]
    Matrix m(a);
    m.mul(b);               // [[4,4],[4,4]]

    a.set(0, 0, 5.0);
    a.print(std::cout);
    std::cout << a.rows() << "x" << a.cols() << std::endl;
    return 0;
}
```

Варианты семейства C

№	Метод	Алгоритм	Тестовый пример
15	double trace() const	След: сумма элементов главной диагонали	`trace([[1,2],[3,4]]) → 5`
16	bool is_symmetric() const	Проверка get(i, j) == get(j, i) для всех пар	`[[1,2],[2,1]] → true; [[1,2],[3,4]] → false`
17	double sum() const	Сумма всех элементов матрицы	`sum([[1,2],[3,4]]) → 10`
18	Matrix pow(size_t n) const	Итеративное умножение (квадратной матрицы)	`A.pow(0) → единичная; A.pow(2) → A²`
19	Matrix submatrix(size_t r, size_t c) const	Копия матрицы без строки r и столбца c	`для [[1,2,3],[4,5,6],[7,8,9]]: submatrix(1,1) → [[1,3],[7,9]]`
20	Matrix rotate90() const	Поворот на 90° по часовой стрелке (вернуть новую матрицу)	`rotate90([[1,2],[3,4]]) → [[3,1],[4,2]]`